

Puntatori a Funzioni

Puntatori a Funzione

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani

Puntatori a Funzioni

Puntatore a Funzione

- Il nome di una funzione
 - In realtà identifica l'indirizzo iniziale in memoria del codice della funzione
- Un puntatore a una funzione contiene l'indirizzo della funzione in memoria
 - Esempio sintassi: `int (*foo) (int, int, int)`
 - Sintassi: mantiene simmetria con dichiarazione della funzione: `int foo (int a, int b, int c);`
- I puntatori alle funzioni possono essere:
 - **Memorizzati** in variabili del tipo opportuno
 - **Passati** a funzioni
 - **Restituiti** da funzioni
 - **Confrontati** tra loro per uguaglianza o disuguaglianza.

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */
```

```
size_t findLastMax(const int a[], size_t n) {
```

```
}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */
```

```
size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]
```

```
}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */

size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]

    for (size_t i = 1; i < n; i++) {

    }

}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */

size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]

    for (size_t i = 1; i < n; i++) {
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i-1]
    }
}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */

size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]

    for (size_t i = 1; i < n; i++) {
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i-1]
        if (a[iMax] <= a[i]) {
            iMax = i;
        }
    }
}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */

size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]

    for (size_t i = 1; i < n; i++) {
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i-1]
        if (a[iMax] <= a[i]) {
            iMax = i;
        }
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i]
    }
}
```

Esempio: `findLastMax()`

- Obiettivo:
 - Trovare l'indice dell'ultima occorrenza del massimo in un array di interi

```
/** @brief Assume: lunghezza(a) >= n >= 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo elemento nella porzione di array a[0,...,n-1]
 */

size_t findLastMax(const int a[], size_t n) {
    size_t iMax = 0; // iMax è l'indice dell'ultima occorrenza del massimo in a[0..0]

    for (size_t i = 1; i < n; i++) {
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i-1]
        if (a[iMax] <= a[i]) {
            iMax = i;
        }
        // iMax è l'indice dell'ultima occorrenza del massimo in a[0..i]
    }
    return iMax;
}
```

find() generica?

- `findLastMax()` restituisce sempre e solo l'indice dell'ultimo elemento più grande
 - E se invece volessimo l'indice del primo elemento più grande?
 - Basterebbe cambiare il predicato in `a[iMax] < a[i]`
 - Il resto della funzione rimane uguale
 - Possiamo definire una funzione generica in base al predicato da usare?
 - Sì, possiamo passare il predicato come puntatore a funzione



Soluzione: `findPar()`



Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine
int less(int x, int y) {return x < y;}    // Una relazione di ordine stretta
```



Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine  
int less(int x, int y) {return x < y;}    // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

Funzioni come predicati alternativi

```
size_t findPar(const int a[], size_t n,  
               int (*compare)(int x, int y)) {
```

52-3

Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine  
int less(int x, int y) {return x < y;}    // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

```
/** @brief Assume: lunghezza(a) >= n >= 1, compare una relazione di ordine  
*/
```

```
size_t findPar(const int a[], size_t n,  
               int (*compare)(int x, int y)) {
```

Puntatore di funzione
come parametro

```
}
```

Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine
int less(int x, int y) {return x < y;}    // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

```
/** @brief Assume: lunghezza(a) >= n >= 1, compare una relazione di ordine
 */
```

```
size_t findPar(const int a[], size_t n,
               int (*compare)(int x, int y)) {
    size_t iMax = 0;
    for (size_t i = 1; i <= n-1; i++) {
        // In alternativa anche compare(a[iMax], a[i])
        if ((*compare)(a[iMax], a[i]))
            iMax = i;
    }
    return iMax;
}
```

Puntatore di funzione
come parametro

Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine
int less(int x, int y) {return x < y;}   // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

```
/** @brief Assume: lunghezza(a) >= n >= 1, compare una relazione di ordine
 */
```

```
size_t findPar(const int a[], size_t n,
               int (*compare)(int x, int y)) {
    size_t iMax = 0;
    for (size_t i = 1; i <= n-1; i++) {
        // In alternativa anche compare(a[iMax], a[i])
        if ((*compare)(a[iMax], a[i]))
            iMax = i;
    }
    return iMax;
}
```

Puntatore di funzione
come parametro

Chiamata alla
funzione puntata da
compare

Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine
int less(int x, int y) {return x < y;}   // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

```
/** @brief Assume: lunghezza(a) >= n >= 1, compare una relazione di ordine
 */
```

```
size_t findPar(const int a[], size_t n,
               int (*compare)(int x, int y)) {
    size_t iMax = 0;
    for (size_t i = 1; i <= n-1; i++) {
        // In alternativa anche compare(a[iMax], a[i])
        if ((*compare)(a[iMax], a[i]))
            iMax = i;
    }
    return iMax;
}
```

Puntatore di funzione
come parametro

Chiamata alla
funzione puntata da
compare

```
void main() {
    ... findLastMaxPar(a, n, lessEq); ... // Restituisce l'indice dell'ultima occorrenza del massimo
    ... findLastMaxPar(a, n, less); ...   // Restituisce l'indice della prima occorrenza del massimo
}
```

Soluzione: findPar()

```
int lessEq(int x, int y) {return x <= y;} // Una relazione di ordine  
int less(int x, int y) {return x < y;} // Una relazione di ordine stretta
```

Funzioni come
predicati alternativi

```
/** @brief Assume: lunghezza(a) >= n >= 1, compare una relazione di ordine  
*/
```

```
size_t findPar(const int a[], size_t n,  
               int (*compare)(int x, int y)) {  
    size_t iMax = 0;  
    for (size_t i = 1; i <= n-1; i++) {  
        // In alternativa anche compare(a[iMax], a[i])  
        if ((*compare)(a[iMax], a[i]))  
            iMax = i;  
    }  
    return iMax;  
}
```

Puntatore di funzione
come parametro

Chiamata alla
funzione puntata da
compare

Passaggio di
lessEq/less come
parametro

```
void main() {  
    ... findLastMaxPar(a, n, lessEq); ... // Restituisce l'indice dell'ultima occorrenza del massimo  
    ... findLastMaxPar(a, n, less); ... // Restituisce l'indice della prima occorrenza del massimo  
}
```